Approximate Q-Learning for Autonomous Navigation in Bomberman

Project 2 — RBE 4701: Artificial Intelligence for Robotics

Amir Sefiyaoui, Jeronimo Ruiz, Vu Nguyen
Team 12
October 9th, 2026

***Abstract*—This report details the design, training, and experimental evaluation of a Bomberman agent built around Approximate Q-Learning. A three-state finite state machine (ADVANCE, BOMB_WAIT, and DANGER) selects which learned linear Q-function scores the agent's 18 possible actions. Every tick, each action is simulated exactly for one step with a lightweight copy of the game rules, exploiting the fact that monsters commit to their move before the character does. Ten features, including a bomb-aware planner distance and a five-tick survival probability computed by Expectimax over the true monster rules, are weighted by parameters learned offline with the approximate Q-learning update. On 200 held-out random seeds per variant the agent wins 100%, 99.0%, 99.5%, 95.0%, and 94.0% of the games in variants 1–5, compared with 46% and 49% in variants 3 and 5 for our first Project 2 version.**

I. Structure of the Approach

The objective of this project is to reach the exit of a Bomberman map in five variants, surviving at least 50% of the runs in each, while using Approximate Q-Learning as part of the solution. The map is eight cells wide; four solid wall rows split it into five compartments that can only be connected by bombing through the walls. Monsters (a random "stupid" monster, a self-preserving monster with detection range 1, and an aggressive one with range 2) start sealed inside these compartments.

Our approach keeps the finite state machine (FSM) structure of our Project 1 agent, but changes what the states do: instead of switching between A* and Expectimax, *every move in every state is chosen by a learned linear Q-function*. The FSM decides which learned weight vector is active.

A. The FSM Evaluation Loop

Every tick, the `do(self, wrld)` method copies the sensed world into a compact simulator state, measures the walking distance (8-connected, through free cells) from each monster to the agent, and selects the state:

$$\text{mode} = \begin{cases} \text{DANGER} & \text{if } \min_{m \in M} d_{walk}(m, \text{agent}) \le 6 \\ \text{BOMB_WAIT} & \text{else if our bomb is live} \\ \text{ADVANCE} & \text{otherwise} \end{cases} \tag{1}$$

where M is the set of monsters. Walking distance (rather than the Chebyshev distance used in Project 1) ignores monsters sealed behind intact walls, while a wall that is already on fire counts as open, since it is about to disappear. The agent then enumerates its 18 actions (the nine moves including standing still, each with or without dropping a bomb), filters them through a safety shield, and executes

$$a^* = \arg\max_a \; Q_{\text{mode}}(s, a) = \arg\max_a \; \mathbf{w}_{\text{mode}} \cdot \mathbf{f}(s, a). \tag{2}$$

Algorithm 1 and Fig. 1 summarize the loop.

Algorithm 1 Per-Tick Decision (`do`)

Require: sensed world `wrld`, learned weights $\mathbf{w}$
$s \leftarrow$ SIM.FROMWORLD(`wrld`)
mode $\leftarrow$ PICKMODE(s) ▷ Eq. (1)
$C \leftarrow \emptyset$
for $(dx, dy) \in 9$ moves, bomb $\in \{\text{no}, \text{yes}\}$ **do**
  $(s', \text{status}) \leftarrow s$.STEP$(dx, dy, \text{bomb})$
  if status $=$ exit **then return** this action
  end if
  if dead **or not** ESCAPEOK(s') **then continue**
  end if
  $C \leftarrow C \cup \{(a, \mathbf{f}(s, a, s'))\}$
end for
return $\arg\max_{(a,\mathbf{f}) \in C} \mathbf{w}_{\text{mode}} \cdot \mathbf{f}$

B. State 1: The ADVANCE State

When no monster can reach the agent within six steps and no bomb is ticking, the agent's job is progress. In practice the learned ADVANCE weights favor actions that shorten the bomb-aware planner distance (Section II-B) and drop a bomb whenever doing so removes a wall on the best route.

C. State 2: The BOMB_WAIT State

While our bomb is live, the agent must stay out of its blast cross until it detonates and the wall fire clears. This state has its own weights: training learned that standing still in a safe cell is better than creeping toward the hole (Section II-D).

D. State 3: The DANGER State

When a monster is within six walking steps, the DANGER weights take over. They weigh the five-tick survival probability and monster proximity much more heavily against progress, so the agent gives up ground to keep its distance from chasers.

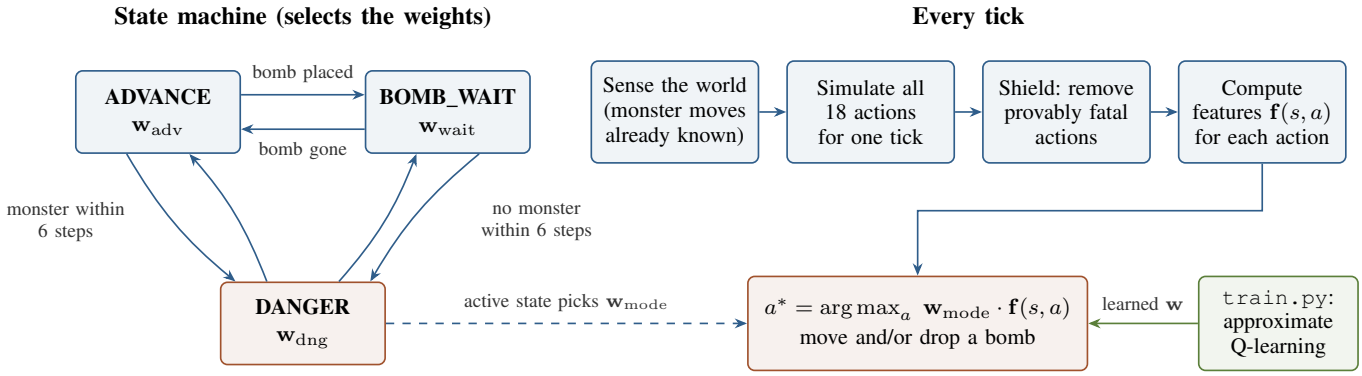


Fig. 1. Left: the state machine selects which learned weight vector scores the actions. Right: the per-tick decision pipeline. The only hand-written rule is the safety shield; the weights are learned offline by `train.py`.

E. The Safety Shield

Before actions are scored, the shield removes every action that the one-tick simulation proves fatal (stepping onto a monster's next cell or into fire, or being caught by a detonation), and every action that leaves the agent inside a bomb's cross from which no path reaches a safe cell before the fuse runs out (a breadth-first search with a time budget). If every action is flagged, the least bad one is taken. Exploration during training also respects the shield.

II. Algorithmic Components and Features

The FSM provides the structure; the quality of the agent comes from the simulator, the features, and how the weights are learned. This section describes each component.

A. Monsters Commit First: An Exact One-Tick Simulator

Reading `RealWorld` revealed that every tick the monsters' `do()` methods run *before* the character's, and the sensed world copies each monster's chosen direction (dx, dy). When the agent decides, it therefore already knows where every monster will step. Our light-weight `Sim` class reproduces `RealWorld.next()` in the same order: fires age and clear walls, bombs tick and explode with the engine's stopping rules (an arm stops at the first wall, monster, or character), monsters move, and finally the character drops its bomb and moves. Two consequences shape the design: whether an action kills the agent this tick is known exactly, and a chaser can only kill the agent if the agent stands next to it when the monster decides. Safety is therefore a question about the *next* decision, which a short lookahead can answer.

B. Bomb-Aware Planner Distance and Bomb Chaining

The *dist* feature is a Dijkstra distance to the exit on the 8-connected grid, with the cost of entering a cell defined as

$$c(x,y) = \begin{cases} 1 & \text{free cell} \\ \max(1, t_{clear}) & \text{wall doomed by a bomb or fire} \\ 16 & \text{intact wall (one bomb cycle)} \end{cases} \tag{3}$$

where t_{clear} is the number of ticks until the wall disappears ($t_{bomb} + 4$ for a bomb with timer t_{bomb}, or the remaining fire time plus one). Because waiting is priced in ticks, the distance keeps falling while the agent waits for a bomb, which gives temporal-difference learning a consistent signal. The *bomb_gain* feature is the drop in this distance that dropping a bomb right now would cause.

An early version charged a flat cost of 2 for a doomed wall; the agent then dropped extra bombs near the end of the map to save a single step at the price of a wait of about 12 ticks. Pricing walls by time removed this behavior. With a blast range of 4 and a wall row every 4 rows, a bomb dropped *inside* a freshly opened hole destroys the next wall four rows below. The learned policy exploits this: in variant 1 it drops bombs at $(0,0)$, $(0,3)$, $(0,7)$, and one near the last wall, and exits in 69 ticks using exactly four bombs (our first version needed 93 ticks).

C. Survival Lookahead via Expectimax

The most important feature, *risk*, is

$$f_{risk}(s,a) = 1 - P_5(s'), \tag{4}$$

where $P_5(s')$ is the probability of still being alive five ticks after reaching s' when the agent plays its best replies. It is computed by Expectimax over the true monster rules. A stupid monster picks uniformly among its non-wall neighbors. A self-preserving monster steps toward the agent if the agent is within its detection range and its current heading is not blocked, otherwise keeps going straight, otherwise picks a random empty cell. The agent's replies are max nodes, the monsters' choices are chance nodes, and bomb blasts follow from the simulator (Algorithm 2).

Four measures keep the search fast. First, monsters that cannot reach the agent within the remaining depth are dropped. Second, random walkers are branched exactly only for the first two levels; deeper, they are frozen in place and every tick spent next to one multiplies the value by $\pi_{frozen} = 1 - 1/n$, its chance of not stepping onto the agent. Third, states are memoized and a max node stops as soon as a reply survives with certainty. Fourth, the search is capped at 30,000 nodes per

TABLE I
FEATURES AND LEARNED WEIGHTS PER STATE (HAND-SET STARTING VALUE IN PARENTHESES WHEN TRAINING CHANGED IT)

Feature	Meaning	ADVANCE	BOMB_WAIT	DANGER
bias	constant 1	54.31 (0)	13.02 (0)	15.50 (0)
dist	planner distance to the exit after the action / 10	−13.71 (−10)	−5.48 (−10)	−5.17 (−10)
bomb_gain	planner distance a bomb dropped now saves / 16	1.86 (2)	0.00	1.00
risk	P(death within 5 ticks), Eq. (4)	−60.00	−60.00	−60.92 (−60)
in_blast	standing in the cross of a live bomb	−1.82 (−0.5)	−1.66 (−2)	−0.94 (−1)
mon_prox	closeness of the nearest monster (within 6 steps)	−2.79 (−2)	−2.01	−3.51 (−4)
chaser_prox	closeness of the nearest chaser (within 10 steps)	−4.36 (−2)	−1.92 (−2)	0.34 (−3)
chaser_near	within detection range +1 of a chaser	−3.01	−3.00	−4.50 (−6)
cornered	fraction of blocked neighboring cells	−1.91 (−0.5)	0.72 (−0.5)	−0.46 (−2)
stay	the action does not move	−0.42 (−0.3)	3.71 (0)	−0.20

Algorithm 2 Survival Lookahead

function SURVIVE(s, d, ℓ) ▷ depth d, level ℓ
  if $d = 0$ **then return** 1
  end if
  $R \leftarrow$ monsters within $d + 1$ of the agent
  freeze random walkers in R if $\ell \geq 2$
  $v \leftarrow 0$
  for each joint monster choice $(p, \mathbf{d})$ of R **do**
    $b \leftarrow 0$
    for each agent move m (farthest first) **do**
      $(s_2, \text{status}) \leftarrow s.\text{STEP}(m \mid \mathbf{d})$
      if status = dead **then** $u \leftarrow 0$
      else if status = exit **then** $u \leftarrow 1$
      else $u \leftarrow \pi_{frozen} \cdot \text{SURVIVE}(s_2, d{-}1, \ell{+}1)$
      end if
      $b \leftarrow \max(b, u)$; **break** if $b = 1$
    end for
    $v \leftarrow v + p \cdot b$
  end for
  return v ▷ memoized
end function

tick. Depth matters: during development, on the 32 variant-4 games that a depth-3 version lost, depth 5 won 26 and depth 7 won 27 at twice the run time, so we use depth 5. Median decision time is 1–5 ms, and the worst case is about 0.2 s.

D. Approximate Q-Learning

The Q-function of each state is linear in the ten features of Table I, $Q_{\text{mode}}(s, a) = \sum_i w_{\text{mode},i} f_i(s, a)$. After each action, the agent applies the approximate Q-learning update with a normalized step for stability:

$$\delta = r + \gamma \max_{a'} Q_{\text{mode}'}(s', a') - Q_{\text{mode}}(s, a), \quad (5)$$

$$w_{\text{mode},i} \leftarrow w_{\text{mode},i} + \frac{\alpha\, \delta\, f_i(s, a)}{1 + \|\mathbf{f}\|^2}, \quad (6)$$

where the next state may belong to a different FSM state. Rewards are -1 per tick, $+100$ for reaching the exit, and -100 for dying; $\gamma = 0.98$, $\alpha = 0.02$, and ϵ-greedy exploration decays from 0.10 to 0.01. Training starts from hand-set weights and runs 1,600 episodes on all five variants with random seeds (harder variants are sampled more often), eight episodes in parallel per batch with their weight changes averaged. It takes about 11 minutes on a 2-core machine (`train.py`).

The changes training made are interpretable. In BOMB_WAIT, standing still became attractive (*stay* $0 \rightarrow 3.71$) and the pull toward the exit weakened (*dist* $-10 \rightarrow -5.48$): waiting out the fuse beats creeping toward the hole. In DANGER the pull toward the exit also halved, so the agent gives up progress for distance from chasers. In ADVANCE the chaser-proximity penalty more than doubled, so the agent keeps away from compartments with chasers before it opens them. The risk weight stays dominant in every state; the bias and distance weights absorb the scale of the value function.

E. Learning from a Naive Start

To check that learning works on its own rather than just inheriting our hand-set weights, we repeated training from a naive vector: *dist* $= -10$ and every other weight 0, which walks to the exit and ignores monsters. We used $\alpha = 0.1$ and otherwise the same procedure. The greedy win rate on variants 2–5 rose from 26% to 88% after 400 episodes and stayed at 88% after 800 (Fig. 2). Without any hint, Q-learning discovered negative weights on risk (-15.8 in DANGER) and on monster proximity (-10.4). It then plateaued and drifted to 81% after 1,600 episodes, below what the hand-set start reaches, so the submitted weights are the ones trained from the hand-set start.

III. EXPERIMENTAL EVALUATION

A. Methodology

The grading rubric defines success in a variant as at least 5 wins in 10 runs. Ten runs give a coarse estimate, so we used `evaluate.py`, which rebuilds each variant exactly as `project2/variantN.py` does (same map, monsters, start positions, and use of the global random seed) and runs the game loop without the display. We report seeds 5000–5199, none of which were used in training; variant 1 has no randomness, so it is a single game. From each measured win rate p we compute the probability of passing the grading test,

$$P_{pass} = \sum_{k=5}^{10} \binom{10}{k} p^k (1-p)^{10-k}. \quad (7)$$

TABLE II
WINS ON 200 HELD-OUT SEEDS PER VARIANT

Var.	First version	Hand-set weights	Trained weights	Avg. ticks first / trained	P_{pass} first / trained
1	1/1	1/1	1/1	93 / 69	100% / 100%
2	183/200	199/200	198/200	114 / 104	>99.9% / >99.9%
3	92/200	199/200	199/200	131 / 82	52.2% / >99.9%
4	139/200	190/200	190/200	138 / 86	94.9% / >99.9%
5	98/200	193/200	188/200	149 / 116	59.8% / >99.9%
Total	513/801	782/801	776/801	–	–

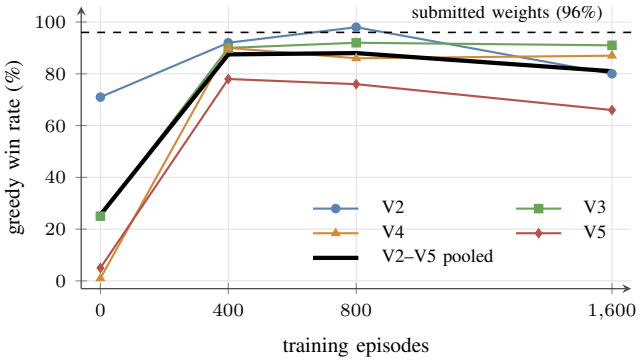


Fig. 2. Learning from a naive start: greedy win rate of the weights saved during training (100 held-out seeds per variant). Dashed: the submitted weights on the same seeds.

In addition, every `variantN.py` file runs and reaches the exit as shipped. Run with ten other seeds each, the shipped variant files won 9/10 (variant 2) and 10/10 (variants 3–5).

B. Results Summary

Table II compares our first Project 2 version (the Project 1 A*/Expectimax agent extended with bombs and a tabular Q-table used only in close combat), the new agent with its hand-set starting weights, and the new agent with the trained weights.

The trained agent lost 25 of 801 games, all to monsters, against 288 losses for the first version, and was never killed by its own bomb. The first version won only 46% of the games in variant 3 and 49% in variant 5, which gave it a substantial chance of failing those variants (worth 60 of the 120 points). The new agent passes the 10-run test with near certainty in all five variants and also finishes faster (Table II).

C. Effect of Training

Training from the hand-set start changed the win counts by at most 5 games out of 200 per variant, which is within sampling noise. It did not make the agent measurably better or worse, but it tuned the trade-offs described in Section II-D. The naive-start experiment (Fig. 2) shows that the learning procedure on its own takes the agent from losing most games in variants 3–5 to winning most of them.

D. Discussion of Remaining Failures

We replayed all 25 losses of the trained agent. Eighteen were caused by the aggressive monster (detection range 2). Either in its own compartment or after following our column of holes upward, it pinned the agent in a compartment only three cells tall, where a chaser moving at the agent's speed cannot be outrun for long, and the five-tick lookahead saw the trap too late. Six were a stupid monster stepping onto the agent: the agent sometimes accepts the roughly 1-in-9 chance of standing next to it when every alternative scores worse, and loses that bet. One loss was caused by the variant-3 monster. A deeper lookahead for chasers, or deliberately luring a chaser into our own blast, are the natural next steps.

IV. CONCLUSION

Replacing the hand-coded A*/Expectimax switch of Project 1 with learned linear Q-functions, selected by a state machine and fed by features from an exact one-tick simulator, produced an agent that wins 94% to 100% of the games in every variant on seeds it never trained on, up from 46%–49% in the hardest variants for our first version. Approximate Q-learning drives every decision: it tuned the trade-offs of the hand-set weights, and starting from a naive weight vector it learned the safety behavior on its own, raising the win rate on variants 2–5 from 26% to 88%.